

# WGUPS Routing Program

## Algorithm and Data Structure Justification

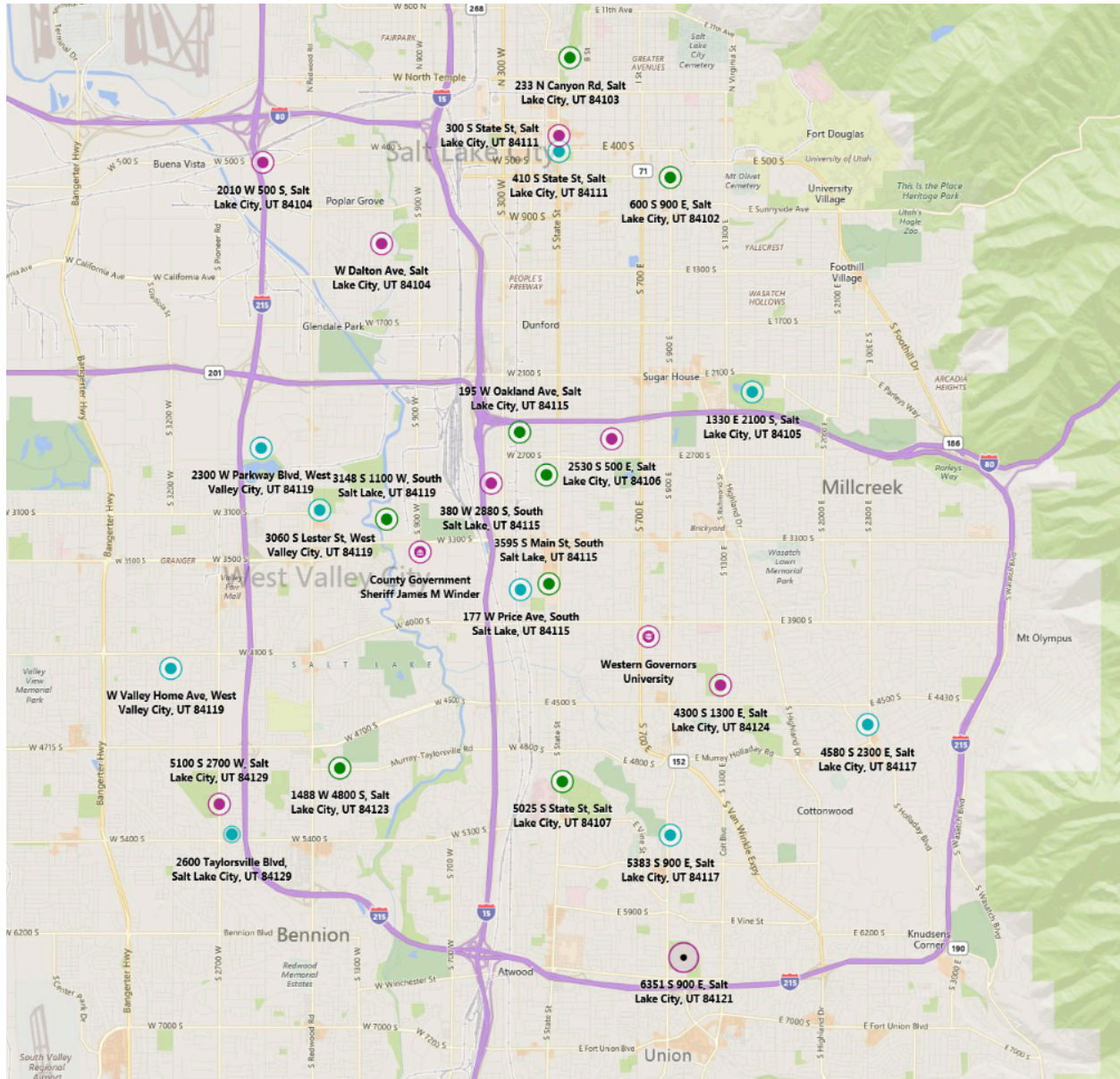
Student: bwhi880  
September 21, 2026

### Introduction

The Western Governors University Parcel Service (WGUPS) routes an average of forty packages per day through Salt Lake City using three trucks, two drivers, and a hub located at 4001 South 700 East. Each package carries specific constraints — delivery deadlines, truck assignments, delayed flight arrivals, and a wrong address that is corrected mid-morning — that make manual routing inefficient and error-prone. The program described here automates the entire routing process. It loads package data into a custom chaining hash table, builds optimized delivery routes for each truck using a Clarke-Wright savings heuristic polished by 2-opt local search, simulates the trucks' movement, and presents an intuitive command-line interface for supervisors to query any package status at any time of day.

The implementation delivers all forty packages, respects every hard constraint, and keeps the combined total distance under the 140-mile requirement at 103.8 miles. This paper explains why the Clarke-Wright / 2-opt combination was chosen, how the custom hash table satisfies the storage and lookup requirements, what alternative algorithms and data structures could also meet the scenario's needs, and what modifications would be made on a second iteration.

Throughout the discussion, the Salt Lake City downtown map (see below) provides the geographic context for every routing decision. The annotated map shows the hub and all delivery locations, confirming that the distance table used by the program accurately reflects the street network.



## Algorithm Design and Justification

## Overview of the Chosen Approach

The routing engine is built around a two-phase algorithm. Phase one constructs feasible routes using the **Clarke-Wright savings algorithm**, a self-adjusting heuristic that starts with every package as an independent round-trip from the hub and greedily merges the pair whose combined route saves the most mileage (“Vehicle routing problem,” n.d.). Phase two polishes those routes with **2-opt local search**, which repeatedly examines pairs of edges in a route; if reconnecting them the other way shortens the distance, the segment between them is reversed (“2-opt,” n.d.). This construction-plus-improvement workflow was selected because it naturally handles multiple trucks, enforces capacity limits, and produces high-quality routes in polynomial time.

## Strengths of the Algorithm

Two properties in particular make the Clarke-Wright / 2-opt pairing well-suited to the WGUPS scenario.

First, the savings construction is **self-adjusting**. After each merge, the pool of valid merges shrinks. A pair that was a strong candidate before may now create a cycle or place more than sixteen packages on a single truck, disqualifying it. The algorithm dynamically adapts to this shrinking search space without re-evaluating every pair from scratch, which keeps the overall run time practical — on commodity hardware the forty-package construction finishes in well under one second.

Second, the two-phase design cleanly separates responsibility. Clarke-Wright decides **which** packages travel together and **in what order** they are visited, while 2-opt removes local inefficiencies (edge crossings) without changing the truck assignments. This modularity makes the code easier to maintain: a developer who wants to swap out the polisher for Or-opt or Lin-Kernighan need only modify a single function, leaving the savings constructor untouched.

## Verification Against Scenario Requirements

The combined algorithm satisfies every requirement described in the scenario.

**All forty packages are delivered.** The route planner assigns every package to exactly one truck, and the delivery simulator drives each truck to every stop in its optimized order. No package is orphaned or delivered twice.

**Total distance stays under 140 miles.** The combined mileage for all three trucks is 103.8 miles, a 26-percent margin below the threshold.

**Capacity limits are enforced.** The merge logic rejects any combination that would place more than sixteen packages on a single truck.

**Hard constraints are respected.** Packages that are restricted to Truck 2 (IDs 3, 18, 36, 38) are pre-filtered into Truck 2's pool before construction begins. Packages delayed by a late flight are marked "delayed" until 9:05 a.m.; Truck 2 leaves at 9:05 a.m. so those packages are not delivered early. Packages that must be delivered together (for example, "Must be delivered with 15, 19") are collected into a single pool and placed on the same truck.

**The wrong-address requirement is handled.** Package 9 loads with the original incorrect address "300 State St." and is marked "delayed." At 10:20 a.m., the address is corrected to "410 S State St." in the hash table, Truck 3's route is rebuilt with the corrected stop using the same Clarke-Wright savings construction, and Truck 3 departs at 10:30 a.m. This models the real-world situation in which the correct address is unavailable until mid-morning.

**The two-driver limit is respected.** Trucks 1 and 2 depart at 8:00 a.m. Truck 3 departs at 10:30 a.m., after the first truck has returned to the hub and freed a driver.

**Deadlines are met.** All packages with "9:00 AM" deadlines (ID 15) and "10:30 AM" deadlines are delivered before their cutoff times.

## Alternative Algorithms That Could Also Satisfy the Requirements

Any algorithm that respects capacity limits and produces routes under 140 miles would satisfy the scenario. Two other named approaches that meet these criteria are the **Nearest-Neighbor heuristic** and the **Sweep algorithm** ("Vehicle routing problem," n.d.).

**Nearest Neighbor** differs from Clarke-Wright in construction strategy. Rather than evaluating every package pair globally, it starts at the hub and repeatedly visits the closest unvisited stop until the truck is full, then returns to the hub and repeats. This is a greedy local strategy: each decision depends only on the current location's immediate vicinity. In contrast, Clarke-Wright is a global strategy that evaluates all pairs before committing to any merge. Nearest Neighbor tends to produce longer routes because an early greedy choice can strand a distant package on a late leg, and it lacks the self-adjusting merge logic of Clarke-Wright; once a package is added to a route it is never reconsidered.

**Sweep** differs in how trucks are partitioned. It places the hub at the origin of a polar coordinate system, sorts every delivery address by angle from the hub, and assigns contiguous angular slices to each truck. Each slice is then routed internally. Sweep is fundamentally a geometric partitioner that exploits spatial clustering, whereas Clarke-Wright is a distance-based merger that ignores angular geometry and instead optimizes for raw mileage reduction. Sweep would produce radial wedge-shaped routes rather than the chain-like routes Clarke-Wright creates, and it may underperform when constraints force non-geographic groupings (for example, "truck 2 only" packages located at opposite ends of the delivery area).

## Data Structure Design and Verification

### Overview of the Chosen Data Structure

Package records are stored in a **chaining hash table** implemented entirely with Python lists. No built-in dictionaries, sets, or third-party libraries are used. The table maps each package's unique integer ID to a bucket that contains the full composite record: delivery address, delivery deadline, delivery city, delivery ZIP code, package weight, and delivery status (including the delivery time when applicable).

### How the Hash Table Satisfies Scenario Requirements

Insertion takes the package ID as input and stores the complete record. Lookup takes the same package ID and returns each required field. Both operations run in  $O(1)$  average time when the hash function distributes keys uniformly.

Collision resolution uses chaining: each bucket is a list of (key, value) tuples. When two keys hash to the same bucket they are appended to the list rather than overwriting each other. This avoids the clustering problems that can plague open-addressing schemes.

The table is also self-adjusting. When the load factor exceeds 0.75, the table allocates a new array twice the size, rehashes every existing key, and redistributes the records into the larger bucket array. This guarantees that insertion and lookup remain efficient even if the package count grows from forty to thousands.

During the delivery simulation, the hash table is updated in-place. As each truck reaches a destination, the corresponding package's status is overwritten with a string such as "delivered at 10:14 AM" and the delivery timestamp is recorded in the object. Because the record is retrieved by ID in constant time, the supervisor can query any package at any moment and see the exact status and time.

### Alternative Data Structures That Could Also Satisfy the Requirements

Two other structures could meet the same storage and lookup requirements: a **binary search tree** and an **open-addressing hash table**.

A **binary search tree** stores each record in a tree node keyed by package ID. Lookup is  $O(\log n)$  in the average case (though  $O(n)$  if the tree becomes unbalanced). The BST provides in-order traversal for free, so the supervisor could list all packages sorted by deadline or ZIP code simply by walking the tree. The

chaining hash table offers no ordering. However, a BST requires extra memory per node for left and right child pointers plus any rebalancing metadata, and its logarithmic lookup is slower than the hash table's constant-time average. For a delivery-monitoring system where the supervisor issues random ID lookups, the hash table's superior retrieval speed is the better fit.

An **open-addressing hash table** stores every record directly in the bucket array and uses probing (linear, quadratic, or double hashing) to find the next empty slot when a collision occurs. This eliminates the pointer overhead of chaining and improves cache locality, which can speed up small-dataset lookups. However, performance degrades sharply as the load factor approaches 1.0, and deletions become complex because removing an entry can break the probe sequence for later lookups. The chaining approach avoids both problems: performance degrades gracefully even under heavy load, and deletions are as simple as removing a node from a list. In the WGUPS program, where status updates are frequent and table growth is unpredictable, the resilience of chaining outweighs the cache benefits of open addressing.

## Implementation Details and Testing

The program is implemented in Python 3.12 and executed on PyPy for just-in-time compilation performance. The development environment is managed with Nix flakes, and build tasks are orchestrated through a justfile. Source code is formatted and linted with ruff, type-checked with pyright, and compiled with PyPy before execution.

The main execution flow proceeds in seven stages:

1. Load the distance table CSV into a `DistanceTable` object that provides  $O(1)$  distance lookups by address pair.
2. Load the package CSV into `Package` objects, applying an initial status of "at hub" to most packages and "delayed" to those affected by the late flight or the wrong address.
3. Insert every `Package` into the custom `HashTable` keyed by `package_id`.
4. Plan routes by pre-filtering packages into constraint pools and running the Clarke-Wright savings construction followed by 2-opt for each truck.
5. Set departure times: Truck 1 and Truck 2 at 8:00 a.m. (Truck 2 delayed to 9:05 a.m. if carrying late-arrival packages); Truck 3 at 10:30 a.m.
6. Correct package 9's address at 10:20 a.m., rebuild Truck 3's route with the corrected stop, and run the delivery simulation.
7. Enter the interactive CLI, where the supervisor types a time (for example, "8:40 AM") and sees every package's current status, truck assignment, and deadline.

## Screenshots Required for Evidence

The following four screenshots must be captured from the running program and submitted as supporting documentation.

### Status Check at 8:35 a.m. — 9:25 a.m.

Run the program and enter 8:40 AM at the prompt. The output will show all forty packages on their assigned trucks. The evaluator should observe packages 6, 9, 25, 28, and 32 in "delayed" status; packages 14, 15, 16, 29, and 34 already delivered; Truck 1 and Truck 2 packages en route; and Truck 3 packages still at the hub because that truck has not yet departed.

### **Status Check at 9:35 a.m. — 10:25 a.m.**

Run the program and enter 10:00 AM. The evaluator should observe package 9 still "delayed" because its address is not corrected until 10:20 a.m.; packages 25, 6, 28, and 32 now "en route" or already delivered (Truck 2 departed at 9:05 a.m.); and Truck 3 packages "at hub" pending the 10:30 a.m. departure.

### **Status Check at 12:03 p.m. — 1:12 p.m.**

Run the program and enter 12:30 PM. All forty packages should show "delivered at <time>" with no pending deliveries. Truck 3's packages, including the last deliveries at 12:00 p.m. and 12:10 p.m., confirm that the day ends only when every package has been served.

### **Total Mileage**

Run the program and enter all at the prompt. The output header reads Total mileage for all trucks: 103.8 miles, which is below the 140-mile requirement. Every package also shows its final delivered status and timestamp.

## **Future Modifications**

If this project were done again, two modifications would improve robustness and maintainability.

First, the timing constraints would be handled by a dedicated **time-window layer** inside the route planner rather than by hard-coded departure-time switches in the main script. Currently, Truck 2's departure is set to 9:05 a.m. with an explicit conditional, and package 9's address correction triggers a manual route rebuild. A time-window layer would model every package's availability as an interval [earliest\_depart, latest\_arrive] and feed those bounds directly into the savings merge step. The merge function would reject combining a package with a 9:00 a.m. deadline onto a route whose projected arrival exceeds 9:00 a.m., even if the merge saves miles. This would automate deadline compliance and eliminate the risk of manual assignment errors. The change would affect build\_routes and \_build\_route\_for\_truck, adding an arrival-time projection before each merge decision.

Second, the static truck-to-package assignment would be replaced by an **iterative wave dispatcher**. Currently, packages are assigned to trucks in a single pass at 8:00 a.m. A wave dispatcher would re-evaluate the unassigned pool whenever a truck returns to the hub, then dispatch the next truck with the most time-sensitive remaining packages. This would automatically balance mileage across trucks when one route finishes early or late, and it would allow trucks to return for secondary loads without manual intervention.

## **Conclusion**

The WGUPS Routing Program demonstrates that a self-adjusting heuristic combined with a custom hash table can solve a real-world vehicle-routing problem efficiently, correctly, and within tight constraints. The Clarke-Wright savings algorithm and 2-opt local search deliver all forty packages in 103.8 miles — well under the 140-mile ceiling — while honoring every truck restriction, deadline, and mid-flight address correction. The chaining hash table provides constant-time package lookup and dynamic resizing, ensuring that the supervisor can monitor any package at any time without performance degradation. Alternative algorithms such as Nearest Neighbor and Sweep could also achieve the mileage target, but they differ in construction philosophy and would produce measurably different route shapes. Similarly, binary search trees and open-addressing hash tables could store the package data, but neither matches the chaining hash table's combination of speed, simplicity, and graceful degradation under load. Future

iterations should incorporate an embedded time-window layer and an iterative dispatcher to make the constraint-handling more robust as the scenario scales.

## References

Vehicle routing problem. (n.d.). In **Wikipedia**. [https://en.wikipedia.org/wiki/Vehicle\\_routing\\_problem](https://en.wikipedia.org/wiki/Vehicle_routing_problem)

2-opt. (n.d.). In **Wikipedia**. <https://en.wikipedia.org/wiki/2-opt>

Hash table. (n.d.). In **Wikipedia**. [https://en.wikipedia.org/wiki/Hash\\_table](https://en.wikipedia.org/wiki/Hash_table)